HARDLY HUMANS · AI CLUB · IIT DHARWAD

AURORA

Team Onboarding and Project Brief

The intent, the implementation path, and the rules that keep us aligned

---

Internal project document

Build useful coding intelligence. Verify what it produces. Spend additional computation only when it is justified.

This document explains not only what we intend to build, but why the project is shaped this way. It is our shared reference for implementation decisions, experiments, scope changes, and technical disagreements.

1 Why This Document Exists

AURORA is ambitious enough that it will be easy to lose weeks to interesting side quests: a new model architecture, a fashionable optimization, a clever agent framework, or a paper that suggests a different direction. Curiosity is welcome; unexamined drift is not.

This document is meant to keep every contributor connected to the same goal and the same implementation logic. It is not a demand that the original plan can never change. It is a demand that changes happen because we have a concrete reason, not because a new idea sounds exciting.

Default rule: follow the agreed implementation path. A proposed change is welcome when it identifies a real problem or a testable opportunity, explains the evidence, and compares the alternative with what we already planned. Until that case is made, the existing plan remains the plan.

This is a living engineering brief. It should be updated when a meaningful decision changes. It should not be rewritten every time someone discovers a new library or reads an interesting paper.

2 The Core Idea: What Is AURORA?

AURORA is a compute-efficient coding intelligence system. It is not simply a language model that returns code, nor is it merely a chat interface wrapped around an application programming interface (API). Our goal is to build a system that can take a programming task, inspect the relevant project, make a bounded change, run checks, interpret failures, and attempt a repair. Over time, we want the system to learn which tasks a compact model can handle and when extra context, tools, or a stronger model are worth the cost.

A typical coding assistant may produce plausible code and leave the human to discover whether it works. AURORA should close that loop:

Understand the task → choose an action → gather context if needed → generate or edit code → run checks → repair, escalate, or finish

The key idea is the loop, not any single model or framework. Code generation is one step in a software-engineering process that also includes context selection, execution, evidence, and revision.

2.1 The problem we are trying to solve

We want to investigate whether a system built around compact coding models, selective retrieval, tools, and execution-based verification can deliver useful coding capability with limited compute and cost. Specifically:

1. **Reliability:** Can compilation, tests, static analysis, and execution feedback help the system catch and repair its mistakes?
2. **Efficiency:** Can we avoid spending expensive model calls on tasks that a smaller model or a simple tool can handle?
3. **Context:** Can we provide the model with relevant repository information without drowning it in irrelevant code?
4. **Improvement:** Can corrections that have been independently verified become training examples that improve the compact model?

These are questions to test, not outcomes to advertise in advance.

3 The Project Philosophy

3.1 Product first, research-grade engineering second, paper only if earned

We will first build a functioning system that someone can observe and test. We will engineer it carefully enough that experiments are reproducible and results are trustworthy. If the results establish a meaningful contribution, we can consider writing a paper. A paper is not the substitute for a working system, and a complicated architecture is not proof of research value.

3.2 Evidence over aesthetic preference

We will not select a design because it is newer, more complex, or sounds more research-oriented. We will select it because it solves a real limitation, or because a controlled experiment shows a useful improvement. A simpler baseline is not embarrassing; it is what lets us understand whether a proposed improvement actually helps.

3.3 Measure the whole system

A model's parameter count, benchmark score, or tokens per second is not sufficient on its own. We care about end-to-end task success, correctness, latency, memory, GPU-hours, API cost, and the amount of human intervention required. Our north-star metric is **verified coding capability per unit of resource**.

3.4 Make claims proportional to evidence

Passing a test suite means the code passed that test suite; it does not prove universal correctness. A model accessed through OpenRouter is an external backend, not a model trained by us. A hypothesis is not a result. We should state limitations plainly and report failed experiments as well as successful ones.

4 The Intended System

The conceptual system has a small number of responsibilities. These are logical components, not a requirement to implement every advanced idea immediately.

- **Agent/orchestrator:** tracks the task, available evidence, actions already attempted, and stopping conditions.
- **Model interface:** gives the agent a consistent way to call different model backends. The rest of the product should not be tied to one vendor.
- **Decision layer:** chooses whether to generate, retrieve, execute, verify, repair, or escalate.
- **Retrieval:** finds relevant repository code, documentation, tests, or API usage when needed.
- **Controlled tools:** allow bounded file operations, searches, and test execution in the intended workspace.
- **Verifier:** turns syntax checks, compilation, tests, and other checks into explicit evidence.
- **Learning pipeline:** may collect suitable, permission-aware, verified examples for later fine-tuning or distillation.

The model should be replaceable without requiring a rewrite of the entire agent. That separation is important because our first working product will use an external model, while our own models will take time to build and evaluate.

5 The Implementation Path: What Comes First and Why

5.1 Stage 1 — Build the working coding agent

The first demonstrator will use an external model through OpenRouter. This is a deliberate engineering shortcut: it lets us validate the agent loop before our own model is ready. It is not a claim that the external model is AURORA's own trained model.

The initial agent should be able to:

1. accept a bounded coding task;
2. inspect a selected repository and identify relevant files;
3. propose and apply a controlled change;
4. run relevant tests or checks in a restricted environment;
5. read the failure output and attempt a targeted repair;
6. show a trace of actions and report what passed, failed, or was not checked.

Exit criterion: repeatable end-to-end demonstrations, with visible tool traces and documented limitations. A demo that only generates code is not enough to establish the intended product loop.

5.2 Stage 2 — Establish our own small-model baseline

Build a reproducible tokenizer, data pipeline, training loop, checkpointing process, and evaluation procedure for a compact decoder-only Transformer trained with next-token prediction. Start with a conventional, understandable baseline. Candidate components such as Rotary Position Embeddings (RoPE), Root Mean Square Layer Normalization (RMSNorm), SwiGLU feed-forward layers, and Grouped Query Attention (GQA) can be considered, but they should not become an excuse to combine every modern technique before a baseline exists.

We must measure actual training throughput, memory use, checkpoint overhead, and evaluation performance before committing to a final model size or token budget. A smaller completed run with useful measurements is more valuable than an ambitious run that cannot be reproduced.

Exit criterion: a reproducible model checkpoint, a clear account of its training budget, and a report of what it can and cannot do.

5.3 Stage 3 — Integrate and compare

Connect our model through the same model interface as the external backend. Evaluate them on the same tasks, with the same test conditions and timeouts. The purpose is to find the capabilities and failure modes of our model honestly, not to arrange a comparison that makes it look better.

Exit criterion: a fair baseline comparison that reports correctness, latency, resource use, and representative failure modes.

5.4 Stage 4 — Add adaptive decisions and selective escalation

Only after we have a working baseline should we test whether the system benefits from choosing between a compact model, retrieval, repair, tools, or a stronger fallback. Start with transparent rules based on observable signals such as task type, test outcomes, repeated failures, and resource cost. A learned decision layer is not automatically better than rules.

Exit criterion: evidence that adaptive decisions improve the quality–cost balance over a single-model or rule-based baseline.

5.5 Stage 5 — Research the ambitious efficiency ideas

Once the baseline is reliable enough to compare against, investigate the higher-risk ideas: Laya as a decision engine, AURORA-Mini/AURORA-Core, learning from verified corrections, semantic-level intervention, speculative decoding, and very-low-bit models. Each experiment should isolate a question. Do not add several of these at once and then guess which one caused a result.

Exit criterion: controlled results showing whether a proposed technique improves capability, efficiency, or both. If it does not, record the result and keep the simpler design.

The stages are ordered by dependency, not by how exciting they sound. We do not need to finish every research direction to have a successful project. We do need a reliable baseline before making claims about improvements.

6 Why We Are Considering These Unconventional Ideas

6.1 AURORA-Mini and AURORA-Core

The long-term concept is a compact workhorse model, **AURORA-Mini**, paired with a stronger fallback, **AURORA-Core**. Mini would handle frequent, routine tasks. Core would be used when the task is harder, Mini repeatedly fails, or the evidence suggests that extra capability is worth the cost. Initially, Core may be an external reference model rather than a model we trained ourselves; that status must be disclosed.

Two models can improve the quality–cost trade-off, but they can also add latency, integration complexity, and routing mistakes. Therefore, the first baseline remains a single-model agent. A two-level setup is justified only if measurements show that selective escalation improves outcomes.

6.2 Laya as a decision engine

Laya is a candidate for the decision-making part of AURORA, not the component responsible for writing source code. Its System-1-style, non-autoregressive decision approach is interesting because a coding agent repeatedly has to choose actions: retrieve more context, run tests, repair, stop, or escalate. In principle, a specialized decision layer could make these choices more cheaply and consistently than asking a generative model to reason from scratch at every step.

The initial system should still use a transparent rule-based policy. We should verify the current Laya implementation, define a suitable interface, and compare it against those rules. If Laya adds overhead or fails to improve the decisions that matter, we should not force it into the product.

6.3 Semantic-level intervention

Token-level generation methods optimize how text is produced. Semantic-level intervention asks a different question: after a meaningful unit such as a function, patch, or test has been produced, should the system continue, retrieve context, run verification, repair, or escalate? The idea is to spend extra capability at the points where it is useful, rather than applying the most expensive process uniformly.

This is a hypothesis, not a guaranteed novel contribution. We need to compare it against fixed retries, single-model generation, and simpler routing. If it increases complexity without increasing verified success or reducing cost, it should be dropped or redesigned.

6.4 Learning from verified corrections

If Mini produces a patch that fails tests and a stronger model produces a corrected patch that passes, that sequence may contain a useful training example. But a fluent answer from a stronger model is not automatically correct. Candidate examples need verification, task relevance checks, permission-aware data handling, and held-out evaluation. We should never train on every generated correction blindly.

The question to test is whether learning from these vetted examples improves Mini on unseen tasks and reduces future intervention without degrading correctness.

6.5 1-bit and 1.58-bit architectures

Very-low-bit model approaches, including BitNet-style methods, are an efficiency research track. The motivation is to reduce model weight memory and potentially improve inference efficiency on suitable hardware. The trade-offs include training stability, quality, specialized kernels, and actual device support. A 1.58-bit model is not simply a conventional trained model whose weights have been rounded after the fact.

We should first have a conventional baseline. Then compare low-bit training or inference with realistic alternatives such as 8-bit and 4-bit quantization where feasible. The relevant result is not the bit count by itself; it is real-device memory, latency, throughput, and coding quality.

6.6 Speculative decoding

Speculative decoding uses a smaller draft model to propose tokens that a larger target model verifies. It is a token-generation acceleration technique, not a replacement for repository retrieval, testing, or semantic decision-making. It may be useful later, but only if our model pairing and runtime make it beneficial. Evaluate it separately from semantic intervention so the results are interpretable.

6.7 Conditional retrieval

Retrieval is useful when a task depends on repository structure, project conventions, external APIs, or existing tests. It can also hurt if it supplies irrelevant context, consumes the context window, and increases latency. Begin with simple search. Add embeddings, reranking, or more complex retrieval only when evidence shows that they improve downstream task success.

7 Compute and Resource Discipline

We have student-project constraints: most laptops do not have high-end discrete GPUs, accelerator access is limited, and cloud spending must remain modest. This means we must be deliberate about every expensive run.

- Use local machines for coding, data inspection, preprocessing, tokenizer experiments, debugging, orchestration, and small tests.
- Use accelerator time only after the pipeline passes a small smoke test and we know what question the run is answering.
- Measure throughput and peak memory before selecting a model size. Do not rely on estimates based only on parameter count.
- Save checkpoints and test that training can resume. Keep the configuration, data version, source revision, and evaluation results with each run.

- Change one major variable at a time when possible. Otherwise, we may spend scarce compute and learn nothing about what helped.
- Keep a budget for evaluation and debugging; do not spend the entire budget on training and leave no time to measure the result.

Resource rule: every expensive experiment must answer a stated question. "Let's see what happens" is not a sufficient plan when the run consumes scarce GPU time or money.

8 How We Will Know Whether Something Works

AURORA requires system-level evaluation, not only model-level metrics. Depending on the stage, useful measurements include:

Dimension	Evidence to record
Correctness	Unit-test pass rate, compilation rate where applicable, regression rate, and known untested behavior.
Task completion	Fraction of tasks completed successfully under fixed conditions.
Repair	Fraction of failures repaired, attempts per task, and repeated failure patterns.
Latency	End-to-end wall-clock time, not only model generation speed.
Resource cost	Tokens, peak memory, GPU-hours, API cost, and relevant human intervention.
Routing	Escalation rate, successful escalations, unnecessary tool calls, and overhead.
Retrieval	Whether the needed information was retrieved, task success with and without retrieval, and added latency.
Learning	Held-out performance before and after fine-tuning, with intervention frequency reported alongside correctness.

Use the same tasks, timeouts, tool permissions, and evaluation procedure when comparing alternatives. Keep evaluation tasks separate from training examples where possible. Report failed runs and limitations; do not select only the best examples for a presentation.

Passing tests is valuable evidence, but it is not a mathematical proof that the program is correct. The system should say which checks passed and which were not run.

9 Rules for Preventing Project Drift

The project should remain open to better ideas while being resistant to distraction. The following rules govern changes to the agreed path.

9.1 The default is to finish the current milestone

Unless a blocker, safety issue, or strong piece of evidence makes the current path untenable, we complete the current milestone and its acceptance checks before introducing a new architectural direction. An idea can be recorded for later without being implemented immediately.

9.2 A new idea must answer five questions

Before replacing an agreed approach, the proposer should explain:

1. **Problem:** What concrete limitation, failure, or opportunity are we addressing?

2. **Evidence:** What observation, measurement, or credible source suggests that the current approach is insufficient?
3. **Alternative:** What exactly would change, and what simpler baseline will it be compared against?
4. **Cost and risk:** What time, compute, integration work, and failure modes will this introduce?
5. **Decision test:** What measurable result would justify adopting it, and what result would make us reject it?

9.3 Use a lightweight change proposal

A proposal does not need to be a formal paper. A short written note is enough, but it should include:

- the current decision and why it was made;
- the proposed change and the reason for reconsidering it;
- the experiment or evidence needed to decide;
- the expected cost, risk, and impact on the milestone;
- a success criterion, a failure criterion, and a fallback plan.

9.4 Separate decisions, hypotheses, and preferences

- **Decision:** an agreed implementation choice that governs current work unless formally revisited.
- **Hypothesis:** an idea that could be true but still requires a test.
- **Preference:** an individual's favored language, framework, or style; useful input, but not sufficient reason to change direction.

A paper or repository may justify investigating an idea. It does not by itself prove that the idea is right for our constraints. Likewise, the founder's opinion is not a substitute for evidence; if a team member produces stronger evidence, we should update the plan.

9.5 Record the decision and move on

Once a decision is made, record the rationale briefly and return to implementation. Reopen it when new evidence materially changes the situation, when the chosen approach fails its acceptance criteria, or when a safety, licensing, or resource constraint requires it. Do not repeatedly reopen settled questions without new information.

10 Engineering and Collaboration Expectations

- **Understand before changing:** read the relevant code, tests, documentation, and issue before editing. Do not make broad changes based on a superficial scan.
- **Make work reviewable:** prefer small, coherent changes with a clear purpose. Explain what changed, why, and how it was tested.
- **Reproduce failures:** when something breaks, record the command, input, environment, expected behavior, actual behavior, and useful error output.

- **Do not hide uncertainty:** say what you do not understand and what you tried. Honest uncertainty is better than a confident but unsupported explanation.
- **Protect credentials and data:** never commit API keys or secrets. Do not send private, restricted, or licensed code to external services without permission.
- **Keep execution bounded:** generated code and commands must run with limited permissions, resource caps, and access restricted to the intended workspace. Destructive operations require explicit review.
- **Review as a team:** disagreements should lead to a test, a written rationale, or a clear decision—not personal arguments or silent divergence.

11 What We Are Not Going to Do

To make the intended scope clear, these are not default priorities for the initial implementation:

- build a huge model just to claim a large parameter count;
- add mixture-of-experts, very-low-bit training, a learned router, and speculative decoding all at once;
- replace a working baseline because a new paper or framework is fashionable;
- treat a successful demo as proof of general capability;
- call an external model our own trained model;
- claim novelty for established techniques such as Transformers, retrieval-augmented generation, tool use, quantization, speculative decoding, or knowledge distillation by themselves;
- spend scarce compute without a hypothesis, an evaluation plan, and a way to interpret the result.

The project may eventually explore any of these directions if the evidence supports it. The point is sequencing and justification, not banning experimentation.

12 What Success Looks Like

Near term: a controlled coding agent can inspect a project, make a bounded change, run checks, expose failures, attempt repair, and show its work.

Medium term: we have a reproducible compact-model training pipeline and a model whose strengths, weaknesses, and resource requirements are measured.

Long term: our model and agent components work together to complete useful coding tasks with a favorable balance of correctness, latency, and cost. Adaptive routing and learning from verified corrections are retained only if they improve that balance.

A paper is a possible outcome if experiments reveal a meaningful result. The primary outcome is a well-engineered, honestly evaluated system and a team that understands what it built.

13 A Note on Using LLMs, Ownership, and Vibe Coding

I actively encourage you to use large language models (LLMs) as learning and development tools. Use them to understand unfamiliar concepts, compare approaches, generate test ideas, explain errors, review code, or help you implement a component. Learning to use these tools well is a valuable engineering skill, and there is no virtue in refusing a useful tool.

However, I will not accept work that has been fully "vibe coded"—generated by an LLM and submitted without the contributor genuinely understanding, validating, and taking ownership of it. If we were willing to accept code that nobody on the team understands, we would not need a group project; one person could simply prompt a model and paste the result.

The standard is not whether you used an LLM. The standard is whether you own the result. For every meaningful contribution, you should be able to:

- explain the purpose and design of the code in your own words;
- justify important choices and identify at least one plausible alternative;
- explain the relevant control flow, data flow, and failure cases;
- run or describe meaningful tests and interpret their results;
- debug the contribution when it fails, rather than asking the model to keep rewriting it blindly;
- identify which parts were LLM-assisted and verify claims, dependencies, and API usage independently.

If you cannot explain a piece of generated code, that is a signal to pause and learn it before submitting it. Ask the model to teach you, write a smaller version yourself, test your understanding, or request a review from another teammate. Do not use generated code as a way to bypass the learning that the project is meant to produce.

Use the tool. Learn from the tool. Verify the tool. Own the result.

AURORA is a team engineering and learning project, not a code-pasting competition.

Closing Principle

The ambition of AURORA is not to build the most complicated system we can describe. It is to build the simplest system that demonstrates useful coding intelligence, then earn the right to make it more sophisticated through evidence. We will stay curious, but we will not confuse novelty with progress. We will change direction when there is a solid reason, record why, and commit to the new path together.

AURORA

Useful code. Verifiable outcomes. Justified computation.